Prenex, Skolem NFs, Horn sentences

Introduction to Logic (CS), Lecture 8

Dan Frumin

University of Groningen

19-20 Oct 2026

Today's Lecture

1. Overview
2. SAT and Logic Programming
3. Prenex and Skolem Normal Form
4. Horn sentences
5. Logic and CS
6. Concluding (Exam Remarks)

Today's Lecture

1. Overview
2. SAT and Logic Programming
3. Prenex and Skolem Normal Form
4. Horn sentences
5. Logic and CS
6. Concluding (Exam Remarks)

Organizational matters

Q and A session; announcement coming later

COVER support lecture: Monday, November 2nd, 11:00-13:00

See the Cover website for the details: `https://svcover.nl/`

This Week

Last week:

- FOL models, validity, satisfiability..
- Rewriting with logical laws with quantifiers.

Today:

- Logic Programming
- Prenex and Skolem normal forms
- Horn sentences & satisfiability in PL

Learning goals: At the end of this week, you are able to

- transform a FOL sentence into prenex and Skolem normal form
- Apply the satisfiability algorithm for Horn sentences in PL

Today's Lecture

1 Overview

2 SAT and Logic Programming

3 Prenex and Skolem Normal Form

4 Horn sentences

5 Logic and CS

6 Concluding (Exam Remarks)

Satisfiability or model existence

An important question in logic is “does a model exist for this formula?” or “is this formula satisfiable?”

- For PL: is there a row in a truth table (situation) making the formula true?
- For FOL: is there a FOL-model (situation) making the formula true?

Satisfiability

Question	In terms of satisfiability
Is there a situation where A is true?	Is A satisfiable?

Satisfiability

Question	**In terms of satisfiability**
Is there a situation where A is true?	Is A satisfiable?
Is A always true?	Is $\neg A$ not satisfiable?

Satisfiability

Question	**In terms of satisfiability**
Is there a situation where A is true?	Is A satisfiable?
Is A always true?	Is $\neg A$ not satisfiable?
Does B follow from A?	Is $A \wedge \neg B$ not satisfiable?

Satisfiability

Question	**In terms of satisfiability**
Is there a situation where A is true?	Is A satisfiable?
Is A always true?	Is $\neg A$ not satisfiable?
Does B follow from A?	Is $A \wedge \neg B$ not satisfiable?
Is B a logical consequence of $A_1, \ldots, A_n$	Is $A_1 \wedge \ldots \wedge A_n \wedge \neg B$ not satisfiable?

Satisfiability

Question	**In terms of satisfiability**
Is there a situation where A is true?	Is A satisfiable?
Is A always true?	Is $\neg A$ not satisfiable?
Does B follow from A?	Is $A \wedge \neg B$ not satisfiable?
Is B a logical consequence of $A_1, \ldots, A_n$	Is $A_1 \wedge \ldots \wedge A_n \wedge \neg B$ not satisfiable?

Many questions can be reformulated into:

Is this propositional logic sentence satisfiable?

Applications of Logic: SAT

SAT itself has many applications in Computer Science

One to highlight: constraints solving in package managers. A set of constraints of a package is a formula in CNF. Each clause in CNF corresponds to one requirement:

- A requires B: $\neg A \vee B$
- require B: $\bot \vee B$, or just B

Applications of Logic: SAT

SAT itself has many applications in Computer Science

One to highlight: constraints solving in package managers. A set of constraints of a package is a formula in CNF. Each clause in CNF corresponds to one requirement:

- A requires B: $\neg A \vee B$
- require B: $\bot \vee B$, or just B
- package A with version $\geq$ 1.1 and $\leq$ 1.3: $A.1.1 \vee A.1.2 \vee A.1.3$

SAT: the satisfiability problem

Definition of SAT

Let S be an arbitrary sentence in propositional logic. Is S satisfiable?

SAT: the satisfiability problem

Definition of SAT

Let S be an arbitrary sentence in propositional logic. Is S satisfiable?

- How to solve SAT?

SAT: the satisfiability problem

Definition of SAT

Let S be an arbitrary sentence in propositional logic. Is S satisfiable?

- How to solve SAT? Generate the truth table for S. Check if is there a row where S gets value T.

SAT: the satisfiability problem

Definition of SAT

Let S be an arbitrary sentence in propositional logic. Is S satisfiable?

- How to solve SAT? Generate the truth table for S. Check if is there a row where S gets value T.
- How much work is this (i.e. how many elementary computation steps)?

SAT: the satisfiability problem

Definition of SAT

Let S be an arbitrary sentence in propositional logic. Is S satisfiable?

- How to solve SAT? Generate the truth table for S. Check if is there a row where S gets value T.
- How much work is this (i.e. how many elementary computation steps)? Order 2^n steps, where n is the amount of atomic propositions in S. With $n > 58$ we have more steps than the age of the universe in seconds.

SAT: the satisfiability problem

Definition of SAT

Let S be an arbitrary sentence in propositional logic. Is S satisfiable?

- How to solve SAT? Generate the truth table for S. Check if is there a row where S gets value T.
- How much work is this (i.e. how many elementary computation steps)? Order 2^n steps, where n is the amount of atomic propositions in S. With $n > 58$ we have more steps than the age of the universe in seconds.
- Can we do better?

SAT: the satisfiability problem

Definition of SAT

Let S be an arbitrary sentence in propositional logic. Is S satisfiable?

- How to solve SAT? Generate the truth table for S. Check if is there a row where S gets value T.
- How much work is this (i.e. how many elementary computation steps)? Order 2^n steps, where n is the amount of atomic propositions in S. With $n > 58$ we have more steps than the age of the universe in seconds.
- Can we do better? There is no faster method known, therefore we call SAT a *hard* problem.

SAT solvers

- There is a variety of SAT solvers, using different tricks to solve SAT faster.
- Some are specialized to particular *theories* (e.g. numerical formulas)
- But for special classes of formulas we can have better results (see: logic programming).







Logic Programming

One (big) application of SAT is logic programming: a programming paradigm based on symbolic logic, in which program is represented by logical formulas: rules (specifications) and a goal.
Examples: Prolog, Datalog, Answer Set Programming

Try Prolog online: `https://swish.swi-prolog.org/`

Example Logic Program

Example situation: Astrid and Jan are parents of Bob, and Bob is a parent of Jens.

$$
\begin{aligned}
&\mathit{Parent}(\mathit{jan}, \mathit{bob}) \wedge \\
&\mathit{Parent}(\mathit{astrid}, \mathit{bob}) \wedge \\
&\mathit{Parent}(\mathit{bob}, \mathit{jens}) \wedge \\
&\forall x \forall y (\mathit{Parent}(x, y) \rightarrow \mathit{Ancestor}(x, y)) \wedge \\
&\forall x \forall y \forall z (\mathit{Parent}(x, y) \wedge \mathit{Ancestor}(y, z) \rightarrow \mathit{Ancestor}(x, z))
\end{aligned}
$$

Questions we can ask:

Example Logic Program

Example situation: Astrid and Jan are parents of Bob, and Bob is a parent of Jens.

$$
\begin{aligned}
&\mathit{Parent}(\mathit{jan}, \mathit{bob}) \wedge \\
&\mathit{Parent}(\mathit{astrid}, \mathit{bob}) \wedge \\
&\mathit{Parent}(\mathit{bob}, \mathit{jens}) \wedge \\
&\forall x \forall y (\mathit{Parent}(x, y) \rightarrow \mathit{Ancestor}(x, y)) \wedge \\
&\forall x \forall y \forall z (\mathit{Parent}(x, y) \wedge \mathit{Ancestor}(y, z) \rightarrow \mathit{Ancestor}(x, z))
\end{aligned}
$$

Questions we can ask:
Is Jan an ancestor of Jens?

$$\mathit{Ancestor}(\mathit{jan}, \mathit{jens})$$

Example Logic Program

Example situation: Astrid and Jan are parents of Bob, and Bob is a parent of Jens.

$$Parent(jan, bob) \wedge$$
$$Parent(astrid, bob) \wedge$$
$$Parent(bob, jens) \wedge$$
$$\forall x \forall y (Parent(x, y) \rightarrow Ancestor(x, y)) \wedge$$
$$\forall x \forall y \forall z (Parent(x, y) \wedge Ancestor(y, z) \rightarrow Ancestor(x, z))$$

Questions we can ask:
Is there an ancestor of Jens?/Who is ancestor of Jens?

$$\exists x Ancestor(x, jens)$$

Example Logic Program

Example situation: Astrid and Jan are parents of Bob, and Bob is a parent of Jens.

$$
\begin{aligned}
&\mathit{Parent}(\mathit{jan}, \mathit{bob}) \wedge \\
&\mathit{Parent}(\mathit{astrid}, \mathit{bob}) \wedge \\
&\mathit{Parent}(\mathit{bob}, \mathit{jens}) \wedge \\
&\forall x \forall y (\mathit{Parent}(x, y) \rightarrow \mathit{Ancestor}(x, y)) \wedge \\
&\forall x \forall y \forall z (\mathit{Parent}(x, y) \wedge \mathit{Ancestor}(y, z) \rightarrow \mathit{Ancestor}(x, z))
\end{aligned}
$$

Questions we can ask:
Is there a common descendant of Astrid and Bob?

$$\exists x (\mathit{Ancestor}(\mathit{astrid}, x) \wedge \mathit{Ancestor}(\mathit{bob}, x))$$

Prolog Example

Database/program:

```
parent(jan, bob).
parent(astrid, bob).
parent(bob, jens).

ancestor(X, Y) :- parent(X, Y).
ancestor(X, Y) :-
  parent(X, Z), ancestor(Z, Y).
```

Queries:

```
  ?- ancestor(jan, jens).
  ?- ancestor(X, jens).
  ?- ancestor(astrid,Y),ancestor(bob,Y).
```

Prolog Example

In general a Prolog program is given by a collection of rules/facts of the form

$$R_i = \forall x_1 .. \forall x_n (P_1(\vec{x}) \wedge \cdots \wedge P_k(\vec{x}) \rightarrow C(\vec{x})),$$

a goal of the form

$$G = \exists y_1 \ldots \exists y_m D(\vec{y}).$$

Question: does the goal follow from the premises?

Equivalent question: is $R_1 \wedge \cdots \wedge R_j \wedge \neg G$ satisfiable?

- Yes $\Rightarrow$ goal does **not** follow from the rules
- No $\Rightarrow$ goal has to follow from the rule

Prolog Example

A Horn clause: only $\forall$ quantifiers in front, particular form of implication

In general a Prolog p a collection of rules/facts of the form

$$R_i = \forall x_1 .. \forall x_n (P_1(\vec{x}) \wedge \cdots \wedge P_k(\vec{x}) \rightarrow C(\vec{x})),$$

a goal of the form

$$G = \exists y_1 \ldots \exists y_m D(\vec{y}).$$

Question: does the goal follow from the premises?

Equivalent question: is $R_1 \wedge \cdots \wedge R_j \wedge \neg G$ satisfiable?

- Yes $\Rightarrow$ goal does **not** follow from the rules
- No $\Rightarrow$ goal has to follow from the rule

Today's Lecture

1 Overview

2 SAT and Logic Programming

3 Prenex and Skolem Normal Form

4 Horn sentences

5 Logic and CS

6 Concluding (Exam Remarks)

Prenex Normal Form: Definition

Informally:
A formula is in **prenex normal form** if all quantifiers are “at the top”.

Prenex Normal Form: Definition

Informally:
A formula is in **prenex normal form** if all quantifiers are "at the top".

Examples: $\forall x \exists y \exists z (P(x, z) \wedge (Q(y, y, x) \rightarrow R(z, y)))$,
$\exists x (R(x, y) \wedge Q(z))$ and $P(a, x) \wedge Q(z)$

Prenex Normal Form: Definition

Informally:
A formula is in **prenex normal form** if all quantifiers are "at the top".

Examples: $\forall x \exists y \exists z (P(x,z) \wedge (Q(y,y,x) \rightarrow R(z,y)))$,
$\exists x (R(x,y) \wedge Q(z))$ and $P(a,x) \wedge Q(z)$

Non-examples: $\forall x \exists y (P(x,z) \wedge \exists z (Q(y,y,x) \rightarrow R(z,y)))$,
$\exists x R(x,y) \rightarrow Q(z)$ and $\neg \forall z Q(z)$

Prenex Normal Form: Definition

Informally:
A formula is in **prenex normal form** if all quantifiers are "at the top".

Examples: $\forall x \exists y \exists z (P(x,z) \wedge (Q(y,y,x) \rightarrow R(z,y)))$,
$\exists x (R(x,y) \wedge Q(z))$ and $P(a,x) \wedge Q(z)$

Non-examples: $\forall x \exists y (P(x,z) \wedge \exists z (Q(y,y,x) \rightarrow R(z,y)))$,
$\exists x R(x,y) \rightarrow Q(z)$ and $\neg \forall z Q(z)$

Definition. A formula is in prenex normal form when it is of the form

$$Q_1\, x_1 \ldots Q_n\, x_n\, A \quad \text{for some } n \geq 0$$

where Q_i is $\forall$ or $\exists$ for $i = 1, \ldots, n$ and A is a formula without quantifiers.

Prenex Normal Form: Definition

Informally:
A formula is in **prenex normal form** if all quantifiers are "at the top".

Examples: $\forall x \exists y \exists z (P(x,z) \wedge (Q(y,y,x) \rightarrow R(z,y)))$,
$\exists x (R(x,y) \wedge Q(z))$ and $P(a,x) \wedge Q(z)$

Non-examples: $\forall x \exists y (P(x,z) \wedge \exists z (Q(y,y,x) \rightarrow R(z,y)))$,
$\exists x R(x,y) \rightarrow Q(z)$ and $\neg \forall z Q(z)$

A *prefix*

Definition. A formula is in prenex normal form when it is of the form

$$Q_1\, x_1 \ldots Q_n\, x_n\, A \quad \textit{for some } n \geq 0$$

where Q_i is $\forall$ or $\exists$ for $i = 1, \ldots, n$ and A is a formula without quantifiers.

Prenex Normal Form: Definition

Informally:
A formula is in **prenex normal form** if all quantifiers are "at the top".

Examples: $\forall x \exists y \exists z (P(x,z) \wedge (Q(y,y,x) \rightarrow R(z,y)))$,
$\exists x (R(x,y) \wedge Q(z))$ and $P(a,x) \wedge Q(z)$

Non-examples: $\forall x \exists y (P(x,z) \wedge \exists z (Q(y,y,x) \rightarrow R(z,y)))$
$\exists x R(x,y) \wedge Q(z)$ and $\forall z Q(z)$

A *prefix*

If $n = 0$ then there are no quantifiers at all

Definition. A formula is in prenex normal form when it is of the form

$$Q_1\, x_1 \ldots Q_n\, x_n\, A \quad \text{for some } n \geq 0$$

where Q_i is $\forall$ or $\exists$ for $i = 1, \ldots, n$ and A is a formula without quantifiers.

Prenex Normal Form: Definition

Informally:
A formula is in **prenex normal form** if all quantifiers are "at the top".

Examples: $\forall x \exists y \exists z (P(x,z) \wedge (Q(y,y,x) \rightarrow R(z,y)))$,
$\exists x (R(x,y) \wedge Q(z))$ and $P(a,x) \wedge Q(z)$

Non-examples: $\forall x \exists y (P(x,z) \wedge \exists z (Q(y,y,x) \rightarrow R(z,y)))$
$\exists x R(x,y) \wedge Q(z)$ and $\forall z Q(z)$

A *prefix*

If $n = 0$ then there are no quantifiers at all

Definition. A formula is in prenex normal form when it is of the form

$$Q_1\, x_1 \ldots Q_n\, x_n\, A \quad \text{for some } n \geq 0$$

where Q_i is $\forall$ or $\exists$ for $i = 1, \ldots, n$ and A is a formula without quantifiers.

Prenex from Latin *praenexus*, something tied or bound up in front

Prenex Normal Form: Applications

Theorem

Every FOL formula is equivalent to a FOL formula in prenex normal form.

Prenex Normal Form: Applications

Theorem

Every FOL formula is equivalent to a FOL formula in prenex normal form.

Some applications:

- **Automated theorem proving** (also application of CNF)
- **Descriptive complexity** (Arithmetical hierarchy):
  A formula $A(x)$ in first-order arithmetic defines a set

$$X_A = \{n \in \mathbb{N} \mid \mathbb{N} \models A(\underline{n})\}$$

  The descriptive complexity of X_A is measured in the number of quantifier alternations in prenex normal form of $A(x)$.

How to Obtain Prenex Normal Form?

Pull the quantifiers to the front,

How to Obtain Prenex Normal Form?

Pull the quantifiers to the front, using logical laws like

$$\begin{aligned}
\neg\forall x P(x) &\Leftrightarrow \exists x \neg P(x) \\
\neg\exists x P(x) &\Leftrightarrow \forall x \neg P(x) \\
\forall x P(x) \wedge Q &\Leftrightarrow \forall x (P(x) \wedge Q) \\
\forall x P(x) \vee Q &\Leftrightarrow \forall x (P(x) \vee Q) \\
\exists x P(x) \wedge Q &\Leftrightarrow \exists x (P(x) \wedge Q) \\
\exists x P(x) \vee Q &\Leftrightarrow \exists x (P(x) \vee Q)
\end{aligned}$$

where x does not occur in Q.

How to Obtain Prenex Normal Form?

Pull the quantifiers to the front, using logical laws like

$$\begin{aligned}
\neg\forall x P(x) &\Leftrightarrow \exists x \neg P(x) \\
\neg\exists x P(x) &\Leftrightarrow \forall x \neg P(x) \\
\forall x P(x) \wedge Q &\Leftrightarrow \forall x (P(x) \wedge Q) \\
\forall x P(x) \vee Q &\Leftrightarrow \forall x (P(x) \vee Q) \\
\exists x P(x) \wedge Q &\Leftrightarrow \exists x (P(x) \wedge Q) \\
\exists x P(x) \vee Q &\Leftrightarrow \exists x (P(x) \vee Q)
\end{aligned}$$

where x does not occur in Q.

- What to do with $Q \wedge \forall x P(x)$?

How to Obtain Prenex Normal Form?

Pull the quantifiers to the front, using logical laws like

$$\begin{aligned}
\neg\forall x P(x) &\Leftrightarrow \exists x \neg P(x) \\
\neg\exists x P(x) &\Leftrightarrow \forall x \neg P(x) \\
\forall x P(x) \wedge Q &\Leftrightarrow \forall x (P(x) \wedge Q) \\
\forall x P(x) \vee Q &\Leftrightarrow \forall x (P(x) \vee Q) \\
\exists x P(x) \wedge Q &\Leftrightarrow \exists x (P(x) \wedge Q) \\
\exists x P(x) \vee Q &\Leftrightarrow \exists x (P(x) \vee Q)
\end{aligned}$$

where x does not occur in Q.

- What to do with $Q \wedge \forall x P(x)$?
  Use commutativity of $\wedge$ and the laws given above, so $\forall x (Q \wedge P(x))$

How to Obtain Prenex Normal Form?

Pull the quantifiers to the front, using logical laws like

$$\begin{aligned}
\neg\forall x P(x) &\Leftrightarrow \exists x \neg P(x) \\
\neg\exists x P(x) &\Leftrightarrow \forall x \neg P(x) \\
\forall x P(x) \wedge Q &\Leftrightarrow \forall x (P(x) \wedge Q) \\
\forall x P(x) \vee Q &\Leftrightarrow \forall x (P(x) \vee Q) \\
\exists x P(x) \wedge Q &\Leftrightarrow \exists x (P(x) \wedge Q) \\
\exists x P(x) \vee Q &\Leftrightarrow \exists x (P(x) \vee Q)
\end{aligned}$$

where x does not occur in Q.

- What to do with $Q \wedge \forall x P(x)$?
  Use commutativity of $\wedge$ and the laws given above, so $\forall x (Q \wedge P(x))$
- What to do with $\forall x P(x) \vee \exists x Q(x)$?

How to Obtain Prenex Normal Form?

Pull the quantifiers to the front, using logical laws like

$$\begin{aligned}
\neg\forall x P(x) &\Leftrightarrow \exists x \neg P(x) \\
\neg\exists x P(x) &\Leftrightarrow \forall x \neg P(x) \\
\forall x P(x) \wedge Q &\Leftrightarrow \forall x (P(x) \wedge Q) \\
\forall x P(x) \vee Q &\Leftrightarrow \forall x (P(x) \vee Q) \\
\exists x P(x) \wedge Q &\Leftrightarrow \exists x (P(x) \wedge Q) \\
\exists x P(x) \vee Q &\Leftrightarrow \exists x (P(x) \vee Q)
\end{aligned}$$

where x does not occur in Q.

- What to do with $Q \wedge \forall x P(x)$?
  Use commutativity of $\wedge$ and the laws given above, so $\forall x (Q \wedge P(x))$
- What to do with $\forall x P(x) \vee \exists x Q(x)$?
  First rename $\exists x Q(x)$ to $\exists y Q(y)$.

How to Obtain Prenex Normal Form?

Pull the quantifiers to the front, using logical laws like

$$\begin{aligned}
\neg\forall x P(x) &\Leftrightarrow \exists x \neg P(x) \\
\neg\exists x P(x) &\Leftrightarrow \forall x \neg P(x) \\
\forall x P(x) \wedge Q &\Leftrightarrow \forall x (P(x) \wedge Q) \\
\forall x P(x) \vee Q &\Leftrightarrow \forall x (P(x) \vee Q) \\
\exists x P(x) \wedge Q &\Leftrightarrow \exists x (P(x) \wedge Q) \\
\exists x P(x) \vee Q &\Leftrightarrow \exists x (P(x) \vee Q)
\end{aligned}$$

where x does not occur in Q.

- What to do with $Q \wedge \forall x P(x)$?
  Use commutativity of $\wedge$ and the laws given above, so $\forall x (Q \wedge P(x))$
- What to do with $\forall x P(x) \vee \exists x Q(x)$?
  First rename $\exists x Q(x)$ to $\exists y Q(y)$. Then:
  $\forall x P(x) \vee \exists y Q(y) \Leftrightarrow$

How to Obtain Prenex Normal Form?

Pull the quantifiers to the front, using logical laws like

$$\begin{aligned}
\neg\forall x P(x) &\Leftrightarrow \exists x \neg P(x) \\
\neg\exists x P(x) &\Leftrightarrow \forall x \neg P(x) \\
\forall x P(x) \wedge Q &\Leftrightarrow \forall x (P(x) \wedge Q) \\
\forall x P(x) \vee Q &\Leftrightarrow \forall x (P(x) \vee Q) \\
\exists x P(x) \wedge Q &\Leftrightarrow \exists x (P(x) \wedge Q) \\
\exists x P(x) \vee Q &\Leftrightarrow \exists x (P(x) \vee Q)
\end{aligned}$$

where x does not occur in Q.

- What to do with $Q \wedge \forall x P(x)$?
  Use commutativity of $\wedge$ and the laws given above, so $\forall x (Q \wedge P(x))$
- What to do with $\forall x P(x) \vee \exists x Q(x)$?
  First rename $\exists x Q(x)$ to $\exists y Q(y)$. Then:
  $\forall x P(x) \vee \exists y Q(y) \Leftrightarrow \forall x \exists y (P(x) \vee Q(y))$ (or $\exists y \forall x (P(x) \vee Q(y))$).

How to Obtain Prenex Normal Form?

1. What to do with $P \rightarrow \forall x Q(x)$? ($x$ does not occur in P)

How to Obtain Prenex Normal Form?

1. What to do with $P \to \forall x Q(x)$? (x does not occur in P)
   Apply $A \to B \Leftrightarrow \neg A \vee B$, so we obtain:

$$P \to \forall x Q(x) \Leftrightarrow \neg P \vee \forall x Q(x)$$

How to Obtain Prenex Normal Form?

1. What to do with $P \to \forall x Q(x)$? (x does not occur in P)
   Apply $A \to B \Leftrightarrow \neg A \vee B$, so we obtain:

$$P \to \forall x Q(x) \Leftrightarrow \neg P \vee \forall x Q(x) \Leftrightarrow \forall x(\neg P \vee Q(x))$$

How to Obtain Prenex Normal Form?

1. What to do with $P \to \forall x Q(x)$? (x does not occur in P)
   Apply $A \to B \Leftrightarrow \neg A \vee B$, so we obtain:

$$P \to \forall x Q(x) \Leftrightarrow \neg P \vee \forall x Q(x) \Leftrightarrow \forall x(\neg P \vee Q(x)) \Leftrightarrow \forall x(P \to Q(x))$$

How to Obtain Prenex Normal Form?

1. What to do with $P \to \forall x Q(x)$? (x does not occur in P)
   Apply $A \to B \Leftrightarrow \neg A \vee B$, so we obtain:

$$P \to \forall x Q(x) \Leftrightarrow \neg P \vee \forall x Q(x) \Leftrightarrow \forall x(\neg P \vee Q(x)) \Leftrightarrow \forall x(P \to Q(x))$$

2. What to do with $\forall x P(x) \to Q$? (x does not occur in Q)

How to Obtain Prenex Normal Form?

1. What to do with $P \to \forall x Q(x)$? (x does not occur in P)
   Apply $A \to B \Leftrightarrow \neg A \vee B$, so we obtain:

$$P \to \forall x Q(x) \Leftrightarrow \neg P \vee \forall x Q(x) \Leftrightarrow \forall x(\neg P \vee Q(x)) \Leftrightarrow \forall x(P \to Q(x))$$

2. What to do with $\forall x P(x) \to Q$? (x does not occur in Q)
   Apply again $A \to B \Leftrightarrow \neg A \vee B$:

$$\forall x P(x) \to Q \Leftrightarrow \neg \forall x P(x) \vee Q$$

How to Obtain Prenex Normal Form?

1. What to do with $P \to \forall x Q(x)$? (x does not occur in P)
   Apply $A \to B \Leftrightarrow \neg A \vee B$, so we obtain:

$$P \to \forall x Q(x) \Leftrightarrow \neg P \vee \forall x Q(x) \Leftrightarrow \forall x(\neg P \vee Q(x)) \Leftrightarrow \forall x(P \to Q(x))$$

2. What to do with $\forall x P(x) \to Q$? (x does not occur in Q)
   Apply again $A \to B \Leftrightarrow \neg A \vee B$:

$$\forall x P(x) \to Q \Leftrightarrow \neg \forall x P(x) \vee Q \Leftrightarrow \exists x \neg P(x) \vee Q$$

How to Obtain Prenex Normal Form?

1. What to do with $P \to \forall x Q(x)$? (x does not occur in P)
   Apply $A \to B \Leftrightarrow \neg A \vee B$, so we obtain:

$$P \to \forall x Q(x) \Leftrightarrow \neg P \vee \forall x Q(x) \Leftrightarrow \forall x(\neg P \vee Q(x)) \Leftrightarrow \forall x(P \to Q(x))$$

2. What to do with $\forall x P(x) \to Q$? (x does not occur in Q)
   Apply again $A \to B \Leftrightarrow \neg A \vee B$:

$$\forall x P(x) \to Q \Leftrightarrow \neg \forall x P(x) \vee Q \Leftrightarrow \exists x \neg P(x) \vee Q \Leftrightarrow \exists x(P(x) \to Q).$$

How to Obtain Prenex Normal Form?

1. What to do with $P \to \forall x Q(x)$? (x does not occur in P)
   Apply $A \to B \Leftrightarrow \neg A \vee B$, so we obtain:

$$P \to \forall x Q(x) \Leftrightarrow \neg P \vee \forall x Q(x) \Leftrightarrow \forall x(\neg P \vee Q(x)) \Leftrightarrow \forall x(P \to Q(x))$$

2. What to do with $\forall x P(x) \to Q$? (x does not occur in Q)
   Apply again $A \to B \Leftrightarrow \neg A \vee B$:

$$\forall x P(x) \to Q \Leftrightarrow \neg \forall x P(x) \vee Q \Leftrightarrow \exists x \neg P(x) \vee Q \Leftrightarrow \exists x(P(x) \to Q).$$

3. What to do with $P \leftrightarrow \forall x Q(x)$? ($x$ does not occur in P)

How to Obtain Prenex Normal Form?

1. What to do with $P \to \forall x Q(x)$? (x does not occur in P)
   Apply $A \to B \Leftrightarrow \neg A \vee B$, so we obtain:

$$P \to \forall x Q(x) \Leftrightarrow \neg P \vee \forall x Q(x) \Leftrightarrow \forall x(\neg P \vee Q(x)) \Leftrightarrow \forall x(P \to Q(x))$$

2. What to do with $\forall x P(x) \to Q$? (x does not occur in Q)
   Apply again $A \to B \Leftrightarrow \neg A \vee B$:

$$\forall x P(x) \to Q \Leftrightarrow \neg \forall x P(x) \vee Q \Leftrightarrow \exists x \neg P(x) \vee Q \Leftrightarrow \exists x(P(x) \to Q).$$

3. What to do with $P \leftrightarrow \forall x Q(x)$? ($x$ does not occur in P)
   Rewrite to $(P \to \forall x Q(x)) \wedge (\forall x Q(x) \to P)$,

How to Obtain Prenex Normal Form?

1. What to do with $P \to \forall x Q(x)$? (x does not occur in P)
   Apply $A \to B \Leftrightarrow \neg A \vee B$, so we obtain:

$$P \to \forall x Q(x) \Leftrightarrow \neg P \vee \forall x Q(x) \Leftrightarrow \forall x(\neg P \vee Q(x)) \Leftrightarrow \forall x(P \to Q(x))$$

2. What to do with $\forall x P(x) \to Q$? (x does not occur in Q)
   Apply again $A \to B \Leftrightarrow \neg A \vee B$:

$$\forall x P(x) \to Q \Leftrightarrow \neg \forall x P(x) \vee Q \Leftrightarrow \exists x \neg P(x) \vee Q \Leftrightarrow \exists x(P(x) \to Q).$$

3. What to do with $P \leftrightarrow \forall x Q(x)$? ($x$ does not occur in P)
   Rewrite to $(P \to \forall x Q(x)) \wedge (\forall x Q(x) \to P)$,
   rename to $(P \to \forall x Q(x)) \wedge (\forall y Q(y) \to P)$, ($y$ does not occur in P)

How to Obtain Prenex Normal Form?

1. What to do with $P \to \forall x Q(x)$? (x does not occur in P)
   Apply $A \to B \Leftrightarrow \neg A \vee B$, so we obtain:

$$P \to \forall x Q(x) \Leftrightarrow \neg P \vee \forall x Q(x) \Leftrightarrow \forall x(\neg P \vee Q(x)) \Leftrightarrow \forall x(P \to Q(x))$$

2. What to do with $\forall x P(x) \to Q$? (x does not occur in Q)
   Apply again $A \to B \Leftrightarrow \neg A \vee B$:

$$\forall x P(x) \to Q \Leftrightarrow \neg \forall x P(x) \vee Q \Leftrightarrow \exists x \neg P(x) \vee Q \Leftrightarrow \exists x(P(x) \to Q).$$

3. What to do with $P \leftrightarrow \forall x Q(x)$? ($x$ does not occur in P)
   Rewrite to $(P \to \forall x Q(x)) \wedge (\forall x Q(x) \to P)$,
   rename to $(P \to \forall x Q(x)) \wedge (\forall y Q(y) \to P)$, ($y$ does not occur in P)
   use 1 and 2 to get: $\forall x \exists y((P \to Q(x)) \wedge (Q(y) \to P))$

Logical laws for prenex normal form ($\forall$)

$$\begin{aligned}
\forall x Q &\Leftrightarrow Q \\
\forall x P(x) &\Leftrightarrow \forall y P(y) \\
\neg \forall x P(x) &\Leftrightarrow \exists x \neg P(x) \\
\forall x P(x) \wedge \forall x R(x) &\Leftrightarrow \forall x (P(x) \wedge R(x)) \\
\forall x P(x) \wedge Q &\Leftrightarrow \forall x (P(x) \wedge Q) \\
Q \wedge \forall x P(x) &\Leftrightarrow \forall x (Q \wedge P(x)) \\
\forall x P(x) \vee Q &\Leftrightarrow \forall x (P(x) \vee Q) \\
Q \vee \forall x P(x) &\Leftrightarrow \forall x (Q \vee P(x)) \\
\forall x P(x) \rightarrow Q &\Leftrightarrow \exists x (P(x) \rightarrow Q) \\
Q \rightarrow \forall x P(x) &\Leftrightarrow \forall x (Q \rightarrow P(x))
\end{aligned}$$

Here x does not occur in Q.

Logical laws for prenex normal form ($\exists$)

$$
\begin{aligned}
\exists x Q &\Leftrightarrow Q \\
\exists x P(x) &\Leftrightarrow \exists y P(y) \\
\neg \exists x P(x) &\Leftrightarrow \forall x \neg P(x) \\
\exists x P(x) \vee \exists x R(x) &\Leftrightarrow \exists x (P(x) \vee R(x)) \\
\exists x P(x) \vee Q &\Leftrightarrow \exists x (P(x) \vee Q) \\
Q \vee \exists x P(x) &\Leftrightarrow \exists x (Q \vee P(x)) \\
\exists x P(x) \wedge Q &\Leftrightarrow \exists x (P(x) \wedge Q) \\
Q \wedge \exists x P(x) &\Leftrightarrow \exists x (Q \wedge P(x)) \\
\exists x P(x) \rightarrow Q &\Leftrightarrow \forall x (P(x) \rightarrow Q) \\
Q \rightarrow \exists x P(x) &\Leftrightarrow \exists x (Q \rightarrow P(x))
\end{aligned}
$$

Here x does not occur in Q.

Prenex normal form: Example

Assume that $A(x)$, $B(y)$, $B(x)$ are quantifier-free.
For example, $A(x) = \mathit{Cube}(x) \wedge \mathit{Small}(x)$.

$$\forall x\, A(x) \vee [\exists y B(y) \rightarrow \forall x B(x)]$$

Prenex normal form: Example

Assume that $A(x)$, $B(y)$, $B(x)$ are quantifier-free.
For example, $A(x) = \mathit{Cube}(x) \wedge \mathit{Small}(x)$.

$$\forall x\, A(x) \vee [\exists y B(y) \rightarrow \forall x B(x)]$$

$$\Leftrightarrow \quad \forall x\, A(x) \vee [\exists y B(y) \rightarrow \forall z B(z)]$$

Prenex normal form: Example

Assume that $A(x)$, $B(y)$, $B(x)$ are quantifier-free.
For example, $A(x) = \mathit{Cube}(x) \wedge \mathit{Small}(x)$.

$$\begin{aligned} & \forall x\, A(x) \vee [\exists y B(y) \rightarrow \forall x B(x)] \\ \Leftrightarrow \quad & \forall x\, A(x) \vee [\exists y B(y) \rightarrow \forall z B(z)] \\ \Leftrightarrow \quad & \forall x\, A(x) \vee \forall y [B(y) \rightarrow \forall z B(z)] \end{aligned}$$

Prenex normal form: Example

Assume that $A(x)$, $B(y)$, $B(x)$ are quantifier-free.
For example, $A(x) = \mathit{Cube}(x) \wedge \mathit{Small}(x)$.

$$\begin{aligned}
& \forall x\, A(x) \vee [\exists y B(y) \rightarrow \forall x B(x)] \\
\Leftrightarrow \quad & \forall x\, A(x) \vee [\exists y B(y) \rightarrow \forall z B(z)] \\
\Leftrightarrow \quad & \forall x\, A(x) \vee \forall y [B(y) \rightarrow \forall z B(z)] \\
\Leftrightarrow \quad & \forall x\, A(x) \vee \forall y \forall z [B(y) \rightarrow B(z)]
\end{aligned}$$

Prenex normal form: Example

Assume that $A(x)$, $B(y)$, $B(x)$ are quantifier-free.
For example, $A(x) = \mathit{Cube}(x) \wedge \mathit{Small}(x)$.

$$
\begin{aligned}
& \forall x\, A(x) \vee [\exists y B(y) \rightarrow \forall x B(x)] \\
\Leftrightarrow \quad & \forall x\, A(x) \vee [\exists y B(y) \rightarrow \forall z B(z)] \\
\Leftrightarrow \quad & \forall x\, A(x) \vee \forall y [B(y) \rightarrow \forall z B(z)] \\
\Leftrightarrow \quad & \forall x\, A(x) \vee \forall y \forall z [B(y) \rightarrow B(z)] \\
\Leftrightarrow \quad & \forall y (\forall x\, A(x) \vee \forall z [B(y) \rightarrow B(z)])
\end{aligned}
$$

Prenex normal form: Example

Assume that $A(x)$, $B(y)$, $B(x)$ are quantifier-free.
For example, $A(x) = \mathit{Cube}(x) \wedge \mathit{Small}(x)$.

$$
\begin{aligned}
& \forall x\, A(x) \vee [\exists y B(y) \rightarrow \forall x B(x)] \\
\Leftrightarrow \quad & \forall x\, A(x) \vee [\exists y B(y) \rightarrow \forall z B(z)] \\
\Leftrightarrow \quad & \forall x\, A(x) \vee \forall y [B(y) \rightarrow \forall z B(z)] \\
\Leftrightarrow \quad & \forall x\, A(x) \vee \forall y \forall z [B(y) \rightarrow B(z)] \\
\Leftrightarrow \quad & \forall y (\forall x\, A(x) \vee \forall z [B(y) \rightarrow B(z)]) \\
\Leftrightarrow \quad & \forall y \forall z (\forall x\, A(x) \vee [B(y) \rightarrow B(z)])
\end{aligned}
$$

Prenex normal form: Example

Assume that $A(x)$, $B(y)$, $B(x)$ are quantifier-free.
For example, $A(x) = \mathit{Cube}(x) \wedge \mathit{Small}(x)$.

$$
\begin{aligned}
& \forall x\, A(x) \vee [\exists y B(y) \rightarrow \forall x B(x)] \\
\Leftrightarrow \quad & \forall x\, A(x) \vee [\exists y B(y) \rightarrow \forall z B(z)] \\
\Leftrightarrow \quad & \forall x\, A(x) \vee \forall y [B(y) \rightarrow \forall z B(z)] \\
\Leftrightarrow \quad & \forall x\, A(x) \vee \forall y \forall z [B(y) \rightarrow B(z)] \\
\Leftrightarrow \quad & \forall y (\forall x\, A(x) \vee \forall z [B(y) \rightarrow B(z)]) \\
\Leftrightarrow \quad & \forall y \forall z (\forall x\, A(x) \vee [B(y) \rightarrow B(z)]) \\
\Leftrightarrow \quad & \forall y \forall z \forall x (A(x) \vee [B(y) \rightarrow B(z)])
\end{aligned}
$$

Skolem Normal Form

Definition.
A FOL sentence is in Skolem normal form if it is in prenex normal form, and has no $\exists$'s.

Applications:

- Automated theorem proving and logic programming (cf. resolution, LPL 18.7, optional)
- Model theory (Löwenheim-Skolem theorem)

Equi-Satisfiability of Skolem Normal Forms

Logical equivalence is too strong of a notion: cannot rewrite $\forall x \exists y(x < y)$ further.

Equi-Satisfiability of Skolem Normal Forms

Logical equivalence is too strong of a notion: cannot rewrite $\forall x \exists y (x < y)$ further.

A weaker notion is *equi-satisfiability*:

Theorem

For every FOL sentence A there is a a (non-equivalent) FOL sentence A' in Skolem normal form such that A is satisfiable iff A' is satisfiable.

Equi-Satisfiability of Skolem Normal Forms

Logical equivalence is too strong of a notion: cannot rewrite $\forall x \exists y (x < y)$ further.

A weaker notion is *equi-satisfiability*:

Theorem

For every FOL sentence A there is a a (non-equivalent) FOL sentence A' in Skolem normal form such that A is satisfiable iff A' is satisfiable.

Simple but a powerful trick; let's look at example.

Equi-Satisfiability of Skolem Normal Forms

Theorem For every FOL sentence A there is a a (non-equivalent) FOL sentence A' in Skolem normal form such that A is satisfiable iff A' is satisfiable.

Consider $A = \forall x \exists y \, \mathit{Loves}(x, y)$ and suppose it has a model $\mathfrak{M}$:

Equi-Satisfiability of Skolem Normal Forms

Theorem For every FOL sentence A there is a a (non-equivalent) FOL sentence A' in Skolem normal form such that A is satisfiable iff A' is satisfiable.

Consider $A = \forall x \exists y \, \mathit{Loves}(x, y)$ and suppose it has a model $\mathfrak{M}$:

- Suppose that $\mathfrak{M} \models \forall x \exists y \, \mathit{Loves}(x, y)$.

Equi-Satisfiability of Skolem Normal Forms

Theorem For every FOL sentence A there is a a (non-equivalent) FOL sentence A' in Skolem normal form such that A is satisfiable iff A' is satisfiable.

Consider $A = \forall x \exists y\, \mathit{Loves}(x, y)$ and suppose it has a model $\mathfrak{M}$:

- Suppose that $\mathfrak{M} \models \forall x \exists y\, \mathit{Loves}(x, y)$.
- Then for every $d \in D$ we can find $d' \in D$ such that $\langle d, d' \rangle \in \mathfrak{M}(\mathit{Loves})$.

Equi-Satisfiability of Skolem Normal Forms

Theorem For every FOL sentence A there is a a (non-equivalent) FOL sentence A' in Skolem normal form such that A is satisfiable iff A' is satisfiable.

Consider $A = \forall x \exists y\, Loves(x, y)$ and suppose it has a model $\mathfrak{M}$:

- Suppose that $\mathfrak{M} \models \forall x \exists y\, Loves(x, y)$.
- Then for every $d \in D$ we can find $d' \in D$ such that $\langle d, d' \rangle \in \mathfrak{M}(Loves)$.
- Turn it into a function $F : D \to D$ such that $\langle d, F(d) \rangle \in \mathfrak{M}(Loves)$ for all $d \in D$

Equi-Satisfiability of Skolem Normal Forms

Theorem For every FOL sentence A there is a a (non-equivalent) FOL sentence A' in Skolem normal form such that A is satisfiable iff A' is satisfiable.

Consider $A = \forall x \exists y\, \mathit{Loves}(x, y)$ and suppose it has a model $\mathfrak{M}$:

- Suppose that $\mathfrak{M} \models \forall x \exists y\, \mathit{Loves}(x, y)$.
- Then for every $d \in D$ we can find $d' \in D$ such that $\langle d, d' \rangle \in \mathfrak{M}(\mathit{Loves})$.
- Turn it into a function $F : D \to D$ such that $\langle d, F(d) \rangle \in \mathfrak{M}(\mathit{Loves})$ for all $d \in D$

Now let $\mathcal{L}'$ be $\mathcal{L}$ extended with a fresh function symbol f (a Skolem function), and extend $\mathfrak{M}$ to an $\mathcal{L}'$-model $\mathfrak{M}'$ by taking $\mathfrak{M}'(f) = F$. Then, $\mathfrak{M}' \models \forall x\, \mathit{Loves}(x, f(x))$.

Skolemization

Main idea: Skolemization exposes dependencies between variables that are implicit in the quantifier order.

Skolemization of a prenex formula:

Starting from the left, replace each $\exists$ by a Skolem function that has the preceding $\forall$-quantified variables as parameters.

Skolemization

Main idea: Skolemization exposes dependencies between variables that are implicit in the quantifier order.

Skolemization of a prenex formula:

> *Starting from the left, replace each $\exists$ by a Skolem function that has the preceding $\forall$-quantified variables as parameters.*

Example:

$$\exists x \forall y \forall z \exists w \forall v A(x, y, z, w, v)$$

Skolemization

Main idea: Skolemization exposes dependencies between variables that are implicit in the quantifier order.

Skolemization of a prenex formula:

> *Starting from the left, replace each $\exists$ by a Skolem function that has the preceding $\forall$-quantified variables as parameters.*

Example:

$\exists x \forall y \forall z \exists w \forall v A(x, y, z, w, v)$

(replace x by a nullary function, i.e. a constant c)

$\forall y \forall z \exists w \forall v A(c, y, z, w, v)$

Skolemization

Main idea: Skolemization exposes dependencies between variables that are implicit in the quantifier order.

Skolemization of a prenex formula:

Starting from the left, replace each $\exists$ by a Skolem function that has the preceding $\forall$-quantified variables as parameters.

Example:

$\exists x \forall y \forall z \exists w \forall v A(x, y, z, w, v)$

(replace x by a nullary function, i.e. a constant c)

$\forall y \forall z \exists w \forall v A(c, y, z, w, v)$

(replace w by a binary function f)

$\forall y \forall z \forall v A(c, y, z, f(y, z), v)$ (in Skolem normal form)

Question on Skolemization

Question: Why can we not just replace all existentially quantified variables by constants? (E.g. in the example on the previous slide, why can we not just replace w by a constant c? Why does w have to be a function of y, z?)

Question on Skolemization

Question: Why can we not just replace all existentially quantified variables by constants? (E.g. in the example on the previous slide, why can we not just replace w by a constant c? Why does w have to be a function of y, z?)

Answer: Doing so will not always produce an equisatisfiable formula.

- Consider the sentence $A = \forall x \exists y (\neg (x = y))$.

Question on Skolemization

Question: Why can we not just replace all existentially quantified variables by constants? (E.g. in the example on the previous slide, why can we not just replace w by a constant c? Why does w have to be a function of y, z?)

Answer: Doing so will not always produce an equisatisfiable formula.

- Consider the sentence $A = \forall x \exists y(\neg(x = y))$.
- If we replace y with a constant, we get: $A' = \forall x(\neg(x = c))$.

Question on Skolemization

Question: Why can we not just replace all existentially quantified variables by constants? (E.g. in the example on the previous slide, why can we not just replace w by a constant c? Why does w have to be a function of y, z?)

Answer: Doing so will not always produce an equisatisfiable formula.

- Consider the sentence $A = \forall x \exists y (\neg(x = y))$.
- If we replace y with a constant, we get: $A' = \forall x (\neg(x = c))$.
- The sentence A is satisfiable (it is easy to come up with a model $\mathfrak{M}$ such that $\mathfrak{M} \models A$.

Question on Skolemization

Question: Why can we not just replace all existentially quantified variables by constants? (E.g. in the example on the previous slide, why can we not just replace w by a constant c? Why does w have to be a function of y, z?)

Answer: Doing so will not always produce an equisatisfiable formula.

- Consider the sentence $A = \forall x \exists y(\neg(x = y))$.
- If we replace y with a constant, we get: $A' = \forall x(\neg(x = c))$.
- The sentence A is satisfiable (it is easy to come up with a model $\mathfrak{M}$ such that $\mathfrak{M} \models A$.
- However, the sentence A' is not satisfiable, because no matter which $\mathfrak{M}$ we take, the formula $\neg(x = c)$ is not satisfied in $\mathfrak{M}$ by the object $d = \mathfrak{M}(c)$.

Question on Skolemization

Question: Why can we not just replace all existentially quantified variables by constants? (E.g. in the example on the previous slide, why can we not just replace w by a constant c? Why does w have to be a function of y, z?)

Answer: Doing so will not always produce an equisatisfiable formula.

- Consider the sentence $A = \forall x \exists y (\neg(x = y))$.
- If we replace y with a constant, we get: $A' = \forall x (\neg(x = c))$.
- The sentence A is satisfiable (it is easy to come up with a model $\mathfrak{M}$ such that $\mathfrak{M} \models A$.
- However, the sentence A' is not satisfiable, because no matter which $\mathfrak{M}$ we take, the formula $\neg(x = c)$ is not satisfied in $\mathfrak{M}$ by the object $d = \mathfrak{M}(c)$.
- Hence, $\mathfrak{M} \not\models A'$ for all $\mathfrak{M}$, i.e., A' is unsatisfiable.

Today's Lecture

1 Overview

2 SAT and Logic Programming

3 Prenex and Skolem Normal Form

4 Horn sentences

5 Logic and CS

6 Concluding (Exam Remarks)

Horn sentences

- A subclass of propositional sentences for which SAT is easy: we only have to check *one* row in the truth table!

Horn sentences

- A subclass of propositional sentences for which SAT is easy: we only have to check *one* row in the truth table!
- **Definition.** A Horn sentence is a sentence $C = C_1 \wedge \cdots \wedge C_n$ such that every clause C_i is of the form
  - $P_1 \wedge \cdots \wedge P_n \rightarrow Q$, or
  - $P_1 \wedge \cdots \wedge P_n \rightarrow \bot$, or
  - $\top \rightarrow Q$ (we can also simply write Q)

  where P's and Q are atomic.

Horn sentences

- A subclass of propositional sentences for which SAT is easy: we only have to check *one* row in the truth table!
- **Definition.** A Horn sentence is a sentence $C = C_1 \wedge \cdots \wedge C_n$ such that every clause C_i is of the form
  - $P_1 \wedge \cdots \wedge P_n \rightarrow Q$, or
  - $P_1 \wedge \cdots \wedge P_n \rightarrow \bot$, or
  - $\top \rightarrow Q$ (we can also simply write Q)

  where P's and Q are atomic.
- **Equivalently:** A Horn sentence is a CNF $C = C_1 \wedge \cdots \wedge C_n$ such that every clause C_i contains *at most one positive literal*.

Conditional form vs CNF

Using the logical laws, we have:

$$P_1 \vee \neg P_2 \vee \cdots \vee \neg P_n \quad \Leftrightarrow P_1 \vee \neg(P_2 \wedge \cdots \wedge P_n) \Leftrightarrow (P_2 \wedge \cdots \wedge P_n) \rightarrow P_1$$

Conditional form vs CNF

Using the logical laws, we have:

$$P_1 \vee \neg P_2 \vee \cdots \vee \neg P_n \quad \Leftrightarrow P_1 \vee \neg(P_2 \wedge \cdots \wedge P_n) \Leftrightarrow (P_2 \wedge \cdots \wedge P_n) \rightarrow P_1$$
$$\neg P_1 \vee \cdots \vee \neg P_n \quad \Leftrightarrow \bot \vee \neg(P_1 \wedge \cdots \wedge P_n) \Leftrightarrow (P_1 \wedge \cdots \wedge P_n) \rightarrow \bot$$

Conditional form vs CNF

Using the logical laws, we have:

$$
\begin{aligned}
P_1 \vee \neg P_2 \vee \cdots \vee \neg P_n &\Leftrightarrow P_1 \vee \neg(P_2 \wedge \cdots \wedge P_n) \Leftrightarrow (P_2 \wedge \cdots \wedge P_n) \rightarrow P_1 \\
\neg P_1 \vee \cdots \vee \neg P_n &\Leftrightarrow \bot \vee \neg(P_1 \wedge \cdots \wedge P_n) \Leftrightarrow (P_1 \wedge \cdots \wedge P_n) \rightarrow \bot \\
P &\Leftrightarrow \top \rightarrow P
\end{aligned}
$$

Conditional form vs CNF

Using the logical laws, we have:

$$\begin{aligned} P_1 \vee \neg P_2 \vee \cdots \vee \neg P_n &\Leftrightarrow P_1 \vee \neg(P_2 \wedge \cdots \wedge P_n) \Leftrightarrow (P_2 \wedge \cdots \wedge P_n) \rightarrow P_1 \\ \neg P_1 \vee \cdots \vee \neg P_n &\Leftrightarrow \bot \vee \neg(P_1 \wedge \cdots \wedge P_n) \Leftrightarrow (P_1 \wedge \cdots \wedge P_n) \rightarrow \bot \\ P &\Leftrightarrow \top \rightarrow P \end{aligned}$$

Hence, we can write every Horn sentence either as a conjunction of implications $(P_1 \wedge \cdots \wedge P_n) \rightarrow Q$ (conditional form), or as $Q \vee \neg P_1 \vee \cdots \vee \neg P_n$ (CNF form) where $P_1, \ldots P_n$ and Q are atoms.

Conditional form vs CNF

Using the logical laws, we have:

$$\begin{aligned} P_1 \vee \neg P_2 \vee \cdots \vee \neg P_n &\Leftrightarrow P_1 \vee \neg(P_2 \wedge \cdots \wedge P_n) \Leftrightarrow (P_2 \wedge \cdots \wedge P_n) \rightarrow P_1 \\ \neg P_1 \vee \cdots \vee \neg P_n &\Leftrightarrow \bot \vee \neg(P_1 \wedge \cdots \wedge P_n) \Leftrightarrow (P_1 \wedge \cdots \wedge P_n) \rightarrow \bot \\ P &\Leftrightarrow \top \rightarrow P \end{aligned}$$

Hence, we can write every Horn sentence either as a conjunction of implications $(P_1 \wedge \cdots \wedge P_n) \rightarrow Q$ (conditional form), or as $Q \vee \neg P_1 \vee \cdots \vee \neg P_n$ (CNF form) where $P_1, \ldots P_n$ and Q are atoms.

Example: The Horn sentence

$$(\neg P_1 \vee P_2 \vee \neg P_4) \wedge \neg P_3 \wedge P_4 \wedge (\neg P_1 \vee \neg P_3)$$

Conditional form vs CNF

Using the logical laws, we have:

$$\begin{aligned} P_1 \vee \neg P_2 \vee \cdots \vee \neg P_n &\Leftrightarrow P_1 \vee \neg(P_2 \wedge \cdots \wedge P_n) \Leftrightarrow (P_2 \wedge \cdots \wedge P_n) \rightarrow P_1 \\ \neg P_1 \vee \cdots \vee \neg P_n &\Leftrightarrow \bot \vee \neg(P_1 \wedge \cdots \wedge P_n) \Leftrightarrow (P_1 \wedge \cdots \wedge P_n) \rightarrow \bot \\ P &\Leftrightarrow \top \rightarrow P \end{aligned}$$

Hence, we can write every Horn sentence either as a conjunction of implications $(P_1 \wedge \cdots \wedge P_n) \rightarrow Q$ (conditional form), or as $Q \vee \neg P_1 \vee \cdots \vee \neg P_n$ (CNF form) where $P_1, \ldots P_n$ and Q are atoms.

Example: The Horn sentence

$$(\neg P_1 \vee P_2 \vee \neg P_4) \wedge \neg P_3 \wedge P_4 \wedge (\neg P_1 \vee \neg P_3)$$

is equivalent to:

$$((P_1 \wedge P_4) \rightarrow P_2)$$

Conditional form vs CNF

Using the logical laws, we have:

$$\begin{aligned} P_1 \vee \neg P_2 \vee \cdots \vee \neg P_n &\Leftrightarrow P_1 \vee \neg(P_2 \wedge \cdots \wedge P_n) \Leftrightarrow (P_2 \wedge \cdots \wedge P_n) \rightarrow P_1 \\ \neg P_1 \vee \cdots \vee \neg P_n &\Leftrightarrow \bot \vee \neg(P_1 \wedge \cdots \wedge P_n) \Leftrightarrow (P_1 \wedge \cdots \wedge P_n) \rightarrow \bot \\ P &\Leftrightarrow \top \rightarrow P \end{aligned}$$

Hence, we can write every Horn sentence either as a conjunction of implications $(P_1 \wedge \cdots \wedge P_n) \rightarrow Q$ (conditional form), or as $Q \vee \neg P_1 \vee \cdots \vee \neg P_n$ (CNF form) where $P_1, \ldots P_n$ and Q are atoms.

Example: The Horn sentence

$$(\neg P_1 \vee P_2 \vee \neg P_4) \wedge \neg P_3 \wedge P_4 \wedge (\neg P_1 \vee \neg P_3)$$

is equivalent to:

$$((P_1 \wedge P_4) \rightarrow P_2) \wedge (P_3 \rightarrow \bot)$$

Conditional form vs CNF

Using the logical laws, we have:

$$\begin{aligned} P_1 \vee \neg P_2 \vee \cdots \vee \neg P_n &\Leftrightarrow P_1 \vee \neg(P_2 \wedge \cdots \wedge P_n) \Leftrightarrow (P_2 \wedge \cdots \wedge P_n) \rightarrow P_1 \\ \neg P_1 \vee \cdots \vee \neg P_n &\Leftrightarrow \bot \vee \neg(P_1 \wedge \cdots \wedge P_n) \Leftrightarrow (P_1 \wedge \cdots \wedge P_n) \rightarrow \bot \\ P &\Leftrightarrow \top \rightarrow P \end{aligned}$$

Hence, we can write every Horn sentence either as a conjunction of implications $(P_1 \wedge \cdots \wedge P_n) \rightarrow Q$ (conditional form), or as $Q \vee \neg P_1 \vee \cdots \vee \neg P_n$ (CNF form) where $P_1, \ldots P_n$ and Q are atoms.

Example: The Horn sentence

$$(\neg P_1 \vee P_2 \vee \neg P_4) \wedge \neg P_3 \wedge P_4 \wedge (\neg P_1 \vee \neg P_3)$$

is equivalent to:

$$((P_1 \wedge P_4) \rightarrow P_2) \wedge (P_3 \rightarrow \bot) \wedge (\top \rightarrow P_4)$$

Conditional form vs CNF

Using the logical laws, we have:

$$\begin{aligned} P_1 \vee \neg P_2 \vee \cdots \vee \neg P_n &\Leftrightarrow P_1 \vee \neg(P_2 \wedge \cdots \wedge P_n) \Leftrightarrow (P_2 \wedge \cdots \wedge P_n) \rightarrow P_1 \\ \neg P_1 \vee \cdots \vee \neg P_n &\Leftrightarrow \bot \vee \neg(P_1 \wedge \cdots \wedge P_n) \Leftrightarrow (P_1 \wedge \cdots \wedge P_n) \rightarrow \bot \\ P &\Leftrightarrow \top \rightarrow P \end{aligned}$$

Hence, we can write every Horn sentence either as a conjunction of implications $(P_1 \wedge \cdots \wedge P_n) \rightarrow Q$ (conditional form), or as $Q \vee \neg P_1 \vee \cdots \vee \neg P_n$ (CNF form) where $P_1, \ldots P_n$ and Q are atoms.

Example: The Horn sentence

$$(\neg P_1 \vee P_2 \vee \neg P_4) \wedge \neg P_3 \wedge P_4 \wedge (\neg P_1 \vee \neg P_3)$$

is equivalent to:

$$((P_1 \wedge P_4) \rightarrow P_2) \wedge (P_3 \rightarrow \bot) \wedge (\top \rightarrow P_4) \wedge ((P_1 \wedge P_3) \rightarrow \bot)$$

Horn sentences in CNF

Which of the following are Horn sentences?

$$P_1 \vee \neg P_2 \vee \neg P_3$$

Horn sentences in CNF

Which of the following are Horn sentences?

$$P_1 \vee \neg P_2 \vee \neg P_3$$

Horn sentences in CNF

Which of the following are Horn sentences?

$$P_1 \vee \neg P_2 \vee \neg P_3$$
$$(\neg P_1 \vee P_2) \wedge \neg P_3 \wedge P_4 \wedge (\neg P_1 \vee \neg P_3)$$

Horn sentences in CNF

Which of the following are Horn sentences?

$$P_1 \vee \neg P_2 \vee \neg P_3$$
$$(\neg P_1 \vee P_2) \wedge \neg P_3 \wedge P_4 \wedge (\neg P_1 \vee \neg P_3)$$

Horn sentences in CNF

Which of the following are Horn sentences?

$$P_1 \vee \neg P_2 \vee \neg P_3$$
$$(\neg P_1 \vee P_2) \wedge \neg P_3 \wedge P_4 \wedge (\neg P_1 \vee \neg P_3)$$
$$\neg P_1 \vee P_2 \vee P_3$$

Horn sentences in CNF

Which of the following are Horn sentences?

$$P_1 \vee \neg P_2 \vee \neg P_3$$
$$(\neg P_1 \vee P_2) \wedge \neg P_3 \wedge P_4 \wedge (\neg P_1 \vee \neg P_3)$$
$$\neg P_1 \vee P_2 \vee P_3$$

Horn sentences in CNF

Which of the following are Horn sentences?

$P_1 \vee \neg P_2 \vee \neg P_3$

$(\neg P_1 \vee P_2) \wedge \neg P_3 \wedge P_4 \wedge (\neg P_1 \vee \neg P_3)$

$\neg P_1 \vee P_2 \vee P_3$

$(\neg P_1 \vee \neg P_2) \wedge \neg P_3$

Horn sentences in CNF

Which of the following are Horn sentences?

$P_1 \vee \neg P_2 \vee \neg P_3$

$(\neg P_1 \vee P_2) \wedge \neg P_3 \wedge P_4 \wedge (\neg P_1 \vee \neg P_3)$

$\neg P_1 \vee P_2 \vee P_3$

$(\neg P_1 \vee \neg P_2) \wedge \neg P_3$

$(\neg P_1 \vee \neg P_3) \wedge (P_1 \vee P_2 \vee \neg P_3)$

Horn sentences in CNF

Which of the following are Horn sentences?

$P_1 \vee \neg P_2 \vee \neg P_3$

$(\neg P_1 \vee P_2) \wedge \neg P_3 \wedge P_4 \wedge (\neg P_1 \vee \neg P_3)$

$\neg P_1 \vee P_2 \vee P_3$

$(\neg P_1 \vee \neg P_2) \wedge \neg P_3$

$(\neg P_1 \vee \neg P_3) \wedge (P_1 \vee P_2 \vee \neg P_3)$

Satisfiability Algorithm for Horn sentences

Consider the following Horn sentence S:

$$
\begin{array}{ll}
 & ((A \wedge D) \rightarrow H) \\
\wedge & (G \rightarrow A) \\
\wedge & (C \rightarrow D) \\
\wedge & ((A \wedge B \wedge E) \rightarrow \bot) \\
\wedge & (\top \rightarrow C) \\
\wedge & ((C \wedge D) \rightarrow G) \\
\wedge & ((A \wedge G) \rightarrow B) \\
\wedge & ((E \wedge F) \rightarrow B)
\end{array}
$$

We try to construct a truth table row that makes S true.

Satisfiability Algorithm for Horn sentences

Consider the following Horn sentence S:

$$
\begin{array}{ll}
 & ((A \wedge D) \rightarrow H) \\
\wedge & (G \rightarrow A) \\
\wedge & (C \rightarrow D) \\
\wedge & ((A \wedge B \wedge E) \rightarrow \bot) \\
\wedge & (\top \rightarrow C) \\
\wedge & ((C \wedge D) \rightarrow G) \\
\wedge & ((A \wedge G) \rightarrow \\
\wedge & ((E \wedge F) \rightarrow
\end{array}
$$

We try to construct a truth table row that makes S true.

	A	B	C	D	E	F	G	H	$\bot$
value:									
step:									

Prepare a truth table with columns for each atom in S and for $\bot$.

Satisfiability Algorithm for Horn sentences

Consider the following Horn sentence S:

$$
\begin{array}{ll}
 & ((A \wedge D) \rightarrow H) \\
\wedge & (G \rightarrow A) \\
\wedge & (C \rightarrow D) \\
\wedge & ((A \wedge B \wedge E) \rightarrow \bot) \\
\wedge & (\top \rightarrow C) \\
\wedge & ((C \wedge D) \rightarrow G) \\
\wedge & ((A \wedge G) \rightarrow B) \\
\wedge & ((E \wedge F) \rightarrow B)
\end{array}
$$

We try to construct a truth table row that makes S true.

	A	B	C	D	E	F	G	H	$\bot$
value:			T						
step:			1						

For each clause $\top \rightarrow A$ in S, assign T to A

Satisfiability Algorithm for Horn sentences

Consider the following Horn sentence S:

$$
\begin{array}{ll}
 & ((A \wedge D) \rightarrow H) \\
\wedge & (G \rightarrow A) \\
\wedge & (C \rightarrow D) \\
\wedge & ((A \wedge B \wedge E) \rightarrow \bot) \\
\wedge & (\top \rightarrow C) \\
\wedge & ((C \wedge D) \rightarrow G) \\
\wedge & ((A \wedge G) \rightarrow B) \\
\wedge & ((E \wedge F) \rightarrow B)
\end{array}
$$

We try to construct a truth table row that makes S true.

	A	B	C	D	E	F	G	H	$\bot$
value:			T	T					
step:			1	2					

Now that we know that C must be T, look for clauses of the form $C \rightarrow A$, assign T to A

Satisfiability Algorithm for Horn sentences

Consider the following Horn sentence S:

$$
\begin{array}{ll}
 & ((A \wedge D) \rightarrow H) \\
\wedge & (G \rightarrow A) \\
\wedge & (C \rightarrow D) \\
\wedge & ((A \wedge B \wedge E) \rightarrow \bot) \\
\wedge & (\top \rightarrow C) \\
\wedge & ((C \wedge D) \rightarrow G) \\
\wedge & ((A \wedge G) \rightarrow B) \\
\wedge & ((E \wedge F) \rightarrow B)
\end{array}
$$

We try to construct a truth table row that makes S true.

	A	B	C	D	E	F	G	H	$\bot$
value:			T	T			T		
step:			1	2			3		

Now we know that both C and D have to be T, then G has to be T as well

Satisfiability Algorithm for Horn sentences

Consider the following Horn sentence S:

$$
\begin{array}{ll}
 & ((A \wedge D) \rightarrow H) \\
\wedge & (G \rightarrow A) \\
\wedge & (C \rightarrow D) \\
\wedge & ((A \wedge B \wedge E) \rightarrow \bot) \\
\wedge & (\top \rightarrow C) \\
\wedge & ((C \wedge D) \rightarrow G) \\
\wedge & ((A \wedge G) \rightarrow B) \\
\wedge & ((E \wedge F) \rightarrow B)
\end{array}
$$

We try to construct a truth table row that makes S true.

	A	B	C	D	E	F	G	H	$\bot$
value:	T		T	T			T		
step:	4		1	2			3		

Continue the same process with all the atoms marked T

Satisfiability Algorithm for Horn sentences

Consider the following Horn sentence S:

$$
\begin{array}{ll}
 & ((A \wedge D) \rightarrow H) \\
\wedge & (G \rightarrow A) \\
\wedge & (C \rightarrow D) \\
\wedge & ((A \wedge B \wedge E) \rightarrow \bot) \\
\wedge & (\top \rightarrow C) \\
\wedge & ((C \wedge D) \rightarrow G) \\
\wedge & ((A \wedge G) \rightarrow B) \\
\wedge & ((E \wedge F) \rightarrow B)
\end{array}
$$

We try to construct a truth table row that makes S true.

	A	B	C	D	E	F	G	H	$\bot$
value:	T	T	T	T			T	T	
step:	4	6	1	2			3	5	

Once the rule no longer applies, we assign F to the remaining columns

Satisfiability Algorithm for Horn sentences

Consider the following Horn sentence S:

$$
\begin{array}{ll}
 & ((A \wedge D) \rightarrow H) \\
\wedge & (G \rightarrow A) \\
\wedge & (C \rightarrow D) \\
\wedge & ((A \wedge B \wedge E) \rightarrow \bot) \\
\wedge & (\top \rightarrow C) \\
\wedge & ((C \wedge D) \rightarrow G) \\
\wedge & ((A \wedge G) \rightarrow B) \\
\wedge & ((E \wedge F) \rightarrow B)
\end{array}
$$

We try to construct a truth table row that makes S true.

	A	B	C	D	E	F	G	H	$\bot$
value:	T	T	T	T	F	F	T	T	F
step:	4	6	1	2			3	5	

The sentence is satisfiable by the assignment h that assigns true to A, B, C, D, G, H, and false to E, F (Note: Modifying h to assign true to F, would also make S true)

Satisfiability Algorithm for Horn sentences

Consider the following Horn sentence S:

$$
\begin{aligned}
& ((A \wedge D) \rightarrow H) \\
\wedge\ & (G \rightarrow A) \\
\wedge\ & (C \rightarrow D) \\
\wedge\ & ((A \wedge B \wedge E) \rightarrow \bot) \\
\wedge\ & (\top \rightarrow C) \\
\wedge\ & ((C \wedge D) \rightarrow G) \\
\wedge\ & ((A \wedge G) \rightarrow B) \\
\wedge\ & ((E \wedge F) \rightarrow B) \\
\wedge\ & (B \rightarrow \bot)
\end{aligned}
$$

We try to construct a truth table row that makes S true.

	A	B	C	D	E	F	G	H	$\bot$
value:	T	T	T	T	F	F	T	T	T
step:	4	6	1	2			3	5	7

Now suppose we had a clause $B \rightarrow \bot$ in S, then at some stage we would assign T to the $\bot$ column

Satisfiability Algorithm for Horn sentences

Consider the following Horn sentence S:

$$
\begin{aligned}
& ((A \wedge D) \rightarrow H) \\
\wedge\ & (G \rightarrow A) \\
\wedge\ & (C \rightarrow D) \\
\wedge\ & ((A \wedge B \wedge E) \rightarrow \bot) \\
\wedge\ & (\top \rightarrow C) \\
\wedge\ & ((C \wedge D) \rightarrow G) \\
\wedge\ & ((A \wedge G) \rightarrow B) \\
\wedge\ & ((E \wedge F) \rightarrow B) \\
\wedge\ & (B \rightarrow \bot)
\end{aligned}
$$

We try to construct a truth table row that makes S true.

	A	B	C	D	E	F	G	H	$\bot$
value:	T	T	T	T	F	F	T	T	T
step:	4	6	1	2			3	5	7

Now suppose we had a clause $B \rightarrow \bot$ in S, then at some stage we would assign T to the $\bot$ column
Then the formula is unsatisfiable

Satisfiability Algorithm for Horn sentences

Let S be a Horn sentence. To check satisfiability of S, we try to construct a truth table row (i.e., truth assignment) that makes S true.

Satisfiability Algorithm for Horn sentences

Let S be a Horn sentence. To check satisfiability of S, we try to construct a truth table row (i.e., truth assignment) that makes S true.

- Make the header of a truth table for S with columns for each atom in S, and also a column for $\bot$.

Satisfiability Algorithm for Horn sentences

Let S be a Horn sentence. To check satisfiability of S, we try to construct a truth table row (i.e., truth assignment) that makes S true.

- Make the header of a truth table for S with columns for each atom in S, and also a column for $\bot$.
- For each conjunct in S of the form $\top \to A$, assign T to A.

Satisfiability Algorithm for Horn sentences

Let S be a Horn sentence. To check satisfiability of S, we try to construct a truth table row (i.e., truth assignment) that makes S true.

- Make the header of a truth table for S with columns for each atom in S, and also a column for $\bot$.
- For each conjunct in S of the form $\top \to A$, assign T to A.
- Repeatedly, apply the following rule:

  If $(P_1 \wedge \ldots \wedge P_k) \to Q$ is a conjunct/clause in S, and $P_1, \ldots, P_k$ are all assigned T, then assign T to A.

Satisfiability Algorithm for Horn sentences

Let S be a Horn sentence. To check satisfiability of S, we try to construct a truth table row (i.e., truth assignment) that makes S true.

- Make the header of a truth table for S with columns for each atom in S, and also a column for $\bot$.
- For each conjunct in S of the form $\top \to A$, assign T to A.
- Repeatedly, apply the following rule:

  If $(P_1 \wedge \ldots \wedge P_k) \to Q$ is a conjunct/clause in S, and $P_1, \ldots, P_k$ are all assigned T, then assign T to A.

until one of these two situations happen:

Satisfiability Algorithm for Horn sentences

Let S be a Horn sentence. To check satisfiability of S, we try to construct a truth table row (i.e., truth assignment) that makes S true.

- Make the header of a truth table for S with columns for each atom in S, and also a column for $\bot$.
- For each conjunct in S of the form $\top \to A$, assign T to A.
- Repeatedly, apply the following rule:

  If $(P_1 \wedge \ldots \wedge P_k) \to Q$ is a conjunct/clause in S, and $P_1, \ldots, P_k$ are all assigned T, then assign T to A.

until one of these two situations happen:

1. the rule assigns T to $\bot$. Then terminate and report: unsatisfiable.

Satisfiability Algorithm for Horn sentences

Let S be a Horn sentence. To check satisfiability of S, we try to construct a truth table row (i.e., truth assignment) that makes S true.

- Make the header of a truth table for S with columns for each atom in S, and also a column for $\bot$.
- For each conjunct in S of the form $\top \to A$, assign T to A.
- Repeatedly, apply the following rule:

  If $(P_1 \wedge \ldots \wedge P_k) \to Q$ is a conjunct/clause in S, and $P_1, \ldots, P_k$ are all assigned T, then assign T to A.

until one of these two situations happen:

1. the rule assigns T to $\bot$. Then terminate and report: unsatisfiable.
2. the rule no longers applies (you have run out of things to do). Then terminate and report satisfiable. Assign F to all atoms that did not yet have a value. The resulting assignment (tt row) makes S true.

Example (CNF form)

Consider the Horn sentence S:

$$(\neg C \vee A) \wedge (\neg C \vee D) \wedge (\neg A \vee B \vee \neg C) \wedge C \wedge (\neg B \vee \neg D) \wedge (\neg E \vee \neg A)$$

We try to construct a satisfying row in the truth table for S

	A	B	C	D	E
value:					
step:					

Example (CNF form)

Consider the Horn sentence S:

$$(\neg C \vee A) \wedge (\neg C \vee D) \wedge (\neg A \vee B \vee \neg C) \wedge C \wedge (\neg B \vee \neg D) \wedge (\neg E \vee \neg A)$$

We try to construct a satisfying row in the truth table for S

	A	B	C	D	E
value:			T		
step:			1		

Example (CNF form)

Consider the Horn sentence S:

$$(\neg C \vee A) \wedge (\neg C \vee D) \wedge (\neg A \vee B \vee \neg C) \wedge C \wedge (\neg B \vee \neg D) \wedge (\neg E \vee \neg A)$$

We try to construct a satisfying row in the truth table for S

	A	B	C	D	E
value:	T		T	T	
step:	2		1	3	

Example (CNF form)

Consider the Horn sentence S:

$$(\neg C \vee A) \wedge (\neg C \vee D) \wedge (\neg A \vee B \vee \neg C) \wedge C \wedge (\neg B \vee \neg D) \wedge (\neg E \vee \neg A)$$

We try to construct a satisfying row in the truth table for S

	A	B	C	D	E
value:	T	T	T	T	
step:	2	4	1	3	

Example (CNF form)

Consider the Horn sentence S:

$$(\neg C \vee A) \wedge (\neg C \vee D) \wedge (\neg A \vee B \vee \neg C) \wedge C \wedge (\neg B \vee \neg D) \wedge (\neg E \vee \neg A)$$

We try to construct a satisfying row in the truth table for S

	A	B	C	D	E
value:	T	T	T	T	
step:	2	4	1	3	

After step 4, the clause $(\neg B \vee \neg D)$ is false. (Analogue of assigning true to $\bot$.) So S is unsatisfiable.

Horn sentences: concluding remarks

- If a Horn sentence S is satisfiable, the Horn satisfiability algorithm computes a satisfying assignment with a *minimal number of atomic sentences assigned to true*. There could be other satisfying assignments, i.e., other rows in the truth table for S that make S true.
- A FOL sentence is in Horn form if it is in Skolem normal form, and the quantifier-free part is in (propositional) Horn form.
- Application: Logic programming (see LPL, exercises 17.27-17.29)

Today's Lecture

1 Overview

2 SAT and Logic Programming

3 Prenex and Skolem Normal Form

4 Horn sentences

5 Logic and CS

6 Concluding (Exam Remarks)

What is logic? (Revisited)

How would you describe logic to your friends in other disciplines or to e.g. your parents?

Keywords:

What is logic? (Revisited)

How would you describe logic to your friends in other disciplines or to e.g. your parents?

Keywords:

- Reasoning
- Abstraction/modeling
- Symbolic/rules/formal, proofs
- Language
- Expressivity
- Symmetry/duality

Today's Lecture

1 Overview

2 SAT and Logic Programming

3 Prenex and Skolem Normal Form

4 Horn sentences

5 Logic and CS

6 Concluding (Exam Remarks)

Exam Material

- All material that has been covered during the course: LPL, Syllabus, slides, exercises. (See BrightSpace)
- There is also a compact overview of LPL chapters in BrightSpace

Dos and Don'ts for the Exam

General:

- Bring your ID
- On the exam sheet you only need to fill in your student number and your name
- Please try to write clearly

On justifications:

- Justifications are always needed in Fitch proofs!
- Justifications not needed when computing prenex, Skolem forms, DNF/CNF.
- Justifications not needed for applying semantic steps in the model, but make sure to be explicit in your reasoning when evaluating quantifiers ("for all ...", "for some ..."). Avoid the use of "for any" (ambiguous)

Dos and Don'ts for the Exam

FO model evaluation:

- Justifications not needed for applying semantic steps
- Remember to state (and argue for) the conclusion: is the original formula true or false?
- Do not write details of choices for existential quantifier that didn't work out. Only show that the good one works.
- Do not write values of variables/constants inside predicates:

  Good: $\mathfrak{M} \models (R(x, y) \wedge \exists z P(x, z))[h[x/2][y/1]]$

  Bad: $\mathfrak{M} \models (R(2, 1) \wedge \exists z P(2, z))[h]$

Dos and Don'ts for the Exam

FO model evaluation:

- Justifications not needed for applying semantic steps
- Remember to state (and argue for) the conclusion: is the original formula true or false?
- Do not write details of choices for existential quantifier that didn't work out. Only show that the good one works.
- Do not write values of variables/constants inside predicates:

  Good: $\mathfrak{M} \models (R(x, y) \wedge \exists z P(x, z))[h[x/2][y/1]]$

  Bad: $\mathfrak{M} \models (R(2, 1) \wedge \exists z P(2, z))[h]$
- Do not mix meta-notation with formal syntax:

  Bad: $2\neg \in \mathfrak{M}(P)$; "$\forall d \in D$", "$\exists d \in D$",...

Preparing for the Exam

Take a look at old exercises on COVER and Brightspace.
One homework/exam exercise can be a source of many other exercises: if you are asked to compute a DNF you can also *a)* construct truth tables to verify that you can an equivalent formula *b)* carry out some rewriting steps in Fitch;

Preparing for the Exam

Take a look at old exercises on COVER and Brightspace.
One homework/exam exercise can be a source of many other exercises: if you are asked to compute a DNF you can also *a)* construct truth tables to verify that you can an equivalent formula *b)* carry out some rewriting steps in Fitch;

or: if you are asked to make a Fitch proof also verify that the argument is a logical consequence via truth tables or models;

Preparing for the Exam

Take a look at old exercises on COVER and Brightspace.
One homework/exam exercise can be a source of many other exercises: if you are asked to compute a DNF you can also *a)* construct truth tables to verify that you can an equivalent formula *b)* carry out some rewriting steps in Fitch;

or: if you are asked to make a Fitch proof also verify that the argument is a logical consequence via truth tables or models;

or: if you are asked to check the validity of an argument also translate it into FOL and try to construct a concrete countermodel;

Preparing for the Exam

Take a look at old exercises on COVER and Brightspace.
One homework/exam exercise can be a source of many other exercises: if you are asked to compute a DNF you can also *a)* construct truth tables to verify that you can an equivalent formula *b)* carry out some rewriting steps in Fitch;

or: if you are asked to make a Fitch proof also verify that the argument is a logical consequence via truth tables or models;

or: if you are asked to check the validity of an argument also translate it into FOL and try to construct a concrete countermodel;

Do try: working in small groups and checking each other work

Wrapping up

Your feedback is much appreciated:

Please fill in the course evaluation questionnaire!

You should received the link soon via email (if not already)

Question session

- Announcement soon, send questions!

Exam

- Final Exam: Thu 5 Nov, 18:15-20:15, see rooster.rug.nl

Good luck with exam preparations!

Going beyond the course: If you liked the material and want to discuss it more or do a self-study, please get into contact with me!